

The Stack Data Structure

A Complete Tutorial

For Undergraduate Computer Science Students

1. Introduction

A stack is one of the most fundamental linear data structures in computer science. It stores a collection of elements and supports only two primary operations for adding and removing data: push (insert) and pop (remove). What makes a stack special is not what it stores, but how it lets you access what's stored — it enforces a strict ordering rule.

LIFO: Last In, First Out

A stack follows the LIFO (Last In, First Out) principle. The last element inserted into the stack is the first one to be removed. You can only ever access the top element — everything else is temporarily "hidden" beneath it.

Contrast this with a queue, which follows FIFO (First In, First Out) — the first element inserted is the first one removed, like a line of people at a ticket counter.

Stack (LIFO)	Queue (FIFO)
5 <- top (pop here)	Front -> [1][2][3][4][5] <- Rear (enqueue here)
4	(dequeue here)
3	
2	
1	
+---	

2. Real-World Analogies

Understanding a stack becomes intuitive once you connect it to everyday situations:

Analogy	How it maps to a Stack
Stack of plates in a cafeteria	You always take the top plate, and new plates are always placed on top. You never pull one from the middle.
Pile of books on a desk	To reach the third book from the bottom, you must first remove the ones above it.
Browser back button	Each page you visit is "pushed" onto a history stack; hitting "back" "pops" you to the previous page.
Undo feature in a text editor	Every action is pushed onto an undo-stack; pressing Ctrl+Z pops the most recent action and reverses it.
Stack of trays / Pringles can	Physically impossible to remove a tray from the bottom without disturbing everything above — this is LIFO.
A pile of laundry	You wash whatever's on top of the pile first, not what's been sitting at the bottom the longest.

These analogies all share one property: access is restricted to one end only. This restriction is precisely what gives the stack its power and predictability in algorithms.

3. The Stack Abstract Data Type (ADT)

As an Abstract Data Type, a stack is defined by its behavior, not its implementation. The core operations are:

Operation	Description	Time Complexity (ideal)
push(x)	Insert element x on top of the stack	O(1)
pop()	Remove and return the top element	O(1)
peek() / top()	Return the top element without removing it	O(1)
isEmpty()	Check whether the stack has no elements	O(1)
isFull()	(Array-based only) Check whether the stack has reached capacity	O(1)
size()	Return the number of elements currently in the stack	O(1)

Key invariant: every operation only touches the top of the stack. There is no way to access, insert, or delete an arbitrary middle element directly — that is what makes a stack a stack, and not just a fancy array.

A stack can be implemented in two common ways: (1) using a static/dynamic array, and (2) using a linked list. Both are covered below with C-style pseudocode.

4. Implementation 1: Array-Based Stack

An array-based stack maintains a fixed-size (or dynamically growable) array and a single integer, `top`, which tracks the index of the topmost element.

```
#define MAX_SIZE 100

struct Stack {
    int arr[MAX_SIZE];
    int top; // index of the top element; -1 means empty
};

// Initialize the stack
void initStack(struct Stack *s) {
    s->top = -1;
}

// Check if stack is empty
int isEmpty(struct Stack *s) {
    return s->top == -1;
}

// Check if stack is full
int isFull(struct Stack *s) {
    return s->top == MAX_SIZE - 1;
}
```

```

}

// Push an element onto the stack
void push(struct Stack *s, int value) {
    if (isFull(s)) {
        printf("Stack Overflow\n");
        return;
    }
    s->top = s->top + 1;
    s->arr[s->top] = value;
}

// Pop the top element off the stack
int pop(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow\n");
        return -1; // sentinel error value
    }
    int value = s->arr[s->top];
    s->top = s->top - 1;
    return value;
}

// Peek at the top element without removing it
int peek(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is Empty\n");
        return -1;
    }
    return s->arr[s->top];
}

```

Advantages: simple, cache-friendly (contiguous memory), fast constant-time operations.

Disadvantages: fixed maximum size (unless you implement dynamic resizing), risk of stack overflow if capacity is exceeded.

5. Implementation 2: Linked-List-Based Stack

A linked-list-based stack avoids the fixed-size limitation. Each node stores data and a pointer to the node below it. The top pointer always references the head of the list — insertion and removal both happen at the head, giving $O(1)$ operations without needing to shift any elements.

```

struct Node {
    int data;
    struct Node *next;
};

```

```

struct Node *top = NULL; // top of stack; NULL means empty

// Check if stack is empty
int isEmpty() {
    return top == NULL;
}

// Push an element onto the stack
void push(int value) {
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Heap Overflow -- cannot allocate memory\n");
        return;
    }
    newNode->data = value;
    newNode->next = top; // new node points to the old top
    top = newNode;      // new node becomes the new top
}

// Pop the top element off the stack
int pop() {
    if (isEmpty()) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    int value = temp->data;
    top = top->next;
    free(temp);
    return value;
}

// Peek at the top element
int peek() {
    if (isEmpty()) {
        printf("Stack is Empty\n");
        return -1;
    }
    return top->data;
}

```

Advantages: grows and shrinks dynamically, no artificial size limit (bounded only by available heap memory).

Disadvantages: extra memory overhead per node (pointer storage), slightly worse cache locality than arrays.

6. Time & Space Complexity

Operation	Array-Based	Linked-List-Based
push	$O(1)$ amortized	$O(1)$
pop	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
Space	$O(n)$, fixed unless resized	$O(n)$, grows dynamically

Both implementations give constant-time operations in the ideal case — the choice between them is a trade-off between memory predictability (array) and flexibility (linked list).

7. Application 1: Recursion and the Call Stack

One of the most important — and often invisible — uses of a stack is in implementing recursion. Every time a function calls itself (or calls another function), the system pushes a new stack frame onto the call stack, containing:

- The function's local variables
- The parameters passed to it
- The return address (where execution should resume once the function completes)

When the function returns, its frame is popped, and control resumes exactly where it left off in the calling function.

Example: Factorial

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

Calling factorial(4) builds up the call stack like this:

```
Push factorial(4) -> waiting for factorial(3)  
Push factorial(3) -> waiting for factorial(2)  
Push factorial(2) -> waiting for factorial(1)  
Push factorial(1) -> waiting for factorial(0)  
Push factorial(0) -> returns 1 immediately
```

Now unwind (pop) in reverse order:

```
Pop factorial(0) -> returns 1  
Pop factorial(1) -> returns  $1 * 1 = 1$   
Pop factorial(2) -> returns  $2 * 1 = 2$   
Pop factorial(3) -> returns  $3 * 2 = 6$   
Pop factorial(4) -> returns  $4 * 6 = 24$ 
```

This is precisely LIFO behavior: the last call made (`factorial(0)`) is the first to return. This is also why deep, unbounded recursion causes a "stack overflow" error — each nested call consumes stack memory, and if recursion goes too deep, the call stack runs out of space.

Key insight: any recursive algorithm can be rewritten iteratively using an explicit stack data structure to simulate the call stack manually — this is a common technique when converting recursive algorithms (like tree traversals or backtracking) into iterative ones.

8. Application 2: Balanced Parentheses Checking

A classic stack application: given a string containing brackets — `()`, `{}`, `[]` — determine whether every opening bracket has a matching, correctly-nested closing bracket.

Algorithm

- Scan the expression left to right, character by character.
- If the character is an opening bracket (`(` , `{` , `[`), push it onto the stack.
- If the character is a closing bracket (`)` , `}` , `]`): if the stack is empty -> unbalanced. Otherwise, pop the top element and check whether it matches the corresponding opening bracket; if not -> unbalanced.
- After scanning the entire string, if the stack is empty -> balanced. If anything remains -> unbalanced (unmatched openers).

Pseudo-code (C-style)

```
int isMatchingPair(char opening, char closing) {
    if (opening == '(' && closing == ')') return 1;
    if (opening == '{' && closing == '}') return 1;
    if (opening == '[' && closing == ']') return 1;
    return 0;
}

int isBalanced(char *expr) {
    struct Stack s;
    initStack(&s);

    for (int i = 0; expr[i] != '\0'; i++) {
        char ch = expr[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            push(&s, ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty(&s)) {
                return 0; // closing bracket with nothing to match
            }
            char top = pop(&s);
        }
    }
    return isEmpty(&s);
}
```

```

    if (!isMatchingPair(top, ch)) {
        return 0; // mismatched pair, e.g. "["
    }
}
// ignore all other characters (letters, digits, operators)
}

return isEmpty(&s); // balanced only if stack is empty at the end
}

```

Trace Example: { [()] }

Char	Action	Stack (top -> right)
{	Push	{
[	Push	{ [
(	Push	{ [(
)	pop, matches (	{ [
]	pop, matches [	{
}	pop, matches {	(empty)

Final stack is empty -> Balanced.

Try ([)] yourself — you'll find that when you hit), the stack top is [, which does not match, so it correctly reports unbalanced. This is exactly how compilers and IDEs detect mismatched brackets in your code.

9. Expression Notations — Background

Before tackling infix/postfix/prefix conversions, it helps to define the three notations clearly.

Notation	Operator Position	Example (A + B)
Infix	Between operands	A + B
Postfix (Reverse Polish)	After operands	A B +
Prefix (Polish)	Before operands	+ A B

Humans naturally read and write infix notation, but it's ambiguous without precedence rules and parentheses (e.g., $A + B * C$ requires knowing that $*$ binds tighter than $+$). Postfix and prefix notations eliminate this ambiguity entirely — no parentheses or precedence rules are ever needed to evaluate them, which is exactly why compilers convert expressions into postfix/prefix form internally before evaluating them.

Operator Precedence and Associativity (needed for conversion)

Operator	Precedence	Associativity
$^$ (exponentiation)	Highest (3)	Right to Left
$*$, $/$	Medium (2)	Left to Right

Operator	Precedence	Associativity
+, -	Lowest (1)	Left to Right

10. Application 3: Infix to Postfix Conversion

This uses the famous Shunting Yard Algorithm (devised by Edsger Dijkstra), which relies entirely on a stack to hold operators temporarily while operands are sent straight to the output.

Algorithm

- Initialize an empty operator stack and an empty output string/list.
- Scan the infix expression left to right, one token at a time.
- If the token is an operand (letter/digit): append it directly to the output.
- If the token is '(': push it onto the stack.
- If the token is ')': pop from the stack and append to output until a '(' is popped (discard both parentheses).
- If the token is an operator (+ - * / ^): while the stack top is not '(' and has higher precedence than (or equal precedence and left-associative with) the current operator, pop it to the output; then push the current operator.
- After the scan, pop any remaining operators from the stack to the output.

Pseudo-code (C-style)

```

int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int isRightAssociative(char op) {
    return op == '^';
}

int isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '^';
}

void infixToPostfix(char *infix, char *postfix) {
    struct Stack s;
    initStack(&s);
    int j = 0; // index into postfix output

    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];

```

```

    if (isalnum(ch)) {           // operand
        postfix[j++] = ch;
    }
    else if (ch == '(') {
        push(&s, ch);
    }
    else if (ch == ')') {
        while (!isEmpty(&s) && peek(&s) != '(') {
            postfix[j++] = pop(&s);
        }
        pop(&s); // discard the '('
    }
    else if (isOperator(ch)) {
        while (!isEmpty(&s) && peek(&s) != '(' &&
            (precedence(peek(&s)) > precedence(ch) ||
            (precedence(peek(&s)) == precedence(ch) && !isRightAssociative(ch)))) {
            postfix[j++] = pop(&s);
        }
        push(&s, ch);
    }
}

while (!isEmpty(&s)) {
    postfix[j++] = pop(&s);
}
postfix[j] = '\0';
}

```

Trace Example: A + B * C

Token	Action	Stack	Output
A	operand -> output	(empty)	A
+	push	+	A
B	operand -> output	+	A B
*	* has higher precedence than +, so just push	+ *	A B
C	operand -> output	+ *	A B C
(end)	pop all remaining	(empty)	A B C * +

Result: A B C * +

A More Complex Example: (A + B) * (C - D)

Token	Action	Stack	Output
(	push	(	

Token	Action	Stack	Output
A	output	(	A
+	push	(+	A
B	output	(+	A B
)	pop until (, discard (	(empty)	A B +
*	push	*	A B +
(	push	* (	A B +
C	output	* (	A B + C
-	push	* (-	A B + C
D	output	* (-	A B + C D
)	pop until (, discard (	*	A B + C D -
(end)	pop all remaining	(empty)	A B + C D - *

Result: A B + C D - *

11. Application 4: Infix to Prefix Conversion

Converting infix to prefix uses a clever three-step trick built on the postfix algorithm above:

Algorithm

- Reverse the infix expression, and while reversing, swap every '(' with ')' and every ')' with '('.
- Convert this reversed-and-swapped expression to postfix using the same algorithm as before, but with one adjustment: for equal-precedence operators, use right-to-left associativity for + - * / during this specific conversion (only ^ naturally stays right-associative). This matters for expressions like A - B - C.
- Reverse the resulting postfix string to get the final prefix expression.

Pseudo-code (C-style)

```
void reverseAndSwap(char *infix, char *reversed) {
    int n = strlen(infix);
    int j = 0;
    for (int i = n - 1; i >= 0; i--) {
        char ch = infix[i];
        if (ch == '(')    reversed[j++] = ')';
        else if (ch == ')') reversed[j++] = '(';
        else             reversed[j++] = ch;
    }
    reversed[j] = '\0';
}

void reverseString(char *str, char *result) {
    int n = strlen(str);
```

```

    for (int i = 0; i < n; i++) {
        result[i] = str[n - 1 - i];
    }
    result[n] = '\0';
}

void infixToPrefix(char *infix, char *prefix) {
    char reversed[200], postfixOfReversed[200];

    reverseAndSwap(infix, reversed);          // Step 1
    infixToPostfix(reversed, postfixOfReversed); // Step 2 (postfix logic above)
    reverseString(postfixOfReversed, prefix);  // Step 3
}

```

Trace Example: $A + B * C$

Step 1 — Reverse and swap: $A + B * C \rightarrow$ reversed $\rightarrow C * B + A$ (no parentheses present, so nothing to swap)

Step 2 — Convert $C * B + A$ to postfix:

Token	Action	Stack	Output
C	output	(empty)	C
*	push	*	C
B	output	*	C B
+	* has higher precedence, pop it, then push +	+	C B *
A	output	+	C B * A
(end)	pop remaining	(empty)	C B * A +

Step 3 — Reverse the result: $C B * A + \rightarrow + A * B C$

Final Prefix Result: $+ A * B C$

Verification: Read prefix $+ A * B C$ as "add A to the product of B and C" — exactly matching the original $A + B * C$.

12. Application 5: Evaluation of Postfix Expression

Once an expression is in postfix form, evaluating it becomes remarkably simple with a single stack — no precedence rules or parentheses are needed at all.

Algorithm

- Scan the postfix expression left to right, one token at a time.
- If the token is an operand, push its numeric value onto the stack.

- If the token is an operator, pop the top two elements: the first popped value is the right operand (b), the second popped value is the left operand (a). Compute a operator b, and push the result back onto the stack.
- After the entire expression is scanned, the stack contains exactly one value — the final result.

Pseudo-code (C-style)

```
int evaluatePostfix(char *postfix) {
    struct Stack s;
    initStack(&s);

    for (int i = 0; postfix[i] != '\0'; i++) {
        char ch = postfix[i];

        if (isdigit(ch)) {
            push(&s, ch - '0'); // convert char digit to int value
        }
        else if (isOperator(ch)) {
            int b = pop(&s); // right operand (popped first!)
            int a = pop(&s); // left operand (popped second!)
            int result;

            switch (ch) {
                case '+': result = a + b; break;
                case '-': result = a - b; break;
                case '*': result = a * b; break;
                case '/': result = a / b; break;
                case '^': result = (int) pow(a, b); break;
                default: result = 0;
            }
            push(&s, result);
        }
    }

    return pop(&s); // final result
}
```

Critical detail: operand order matters for non-commutative operators (-, /, ^). The second popped value (a) is always the left operand, and the first popped value (b) is the right operand: a - b, never b - a.

Trace Example: 6 5 2 3 + 8 * + 3 + *

Token	Action	Stack (bottom -> top)
6	push	6
5	push	6 5

Token	Action	Stack (bottom -> top)
2	push	6 5 2
3	push	6 5 2 3
+	pop 3,2 -> 2+3=5, push	6 5 5
8	push	6 5 5 8
*	pop 8,5 -> 5*8=40, push	6 5 40
+	pop 40,5 -> 5+40=45, push	6 45
3	push	6 45 3
+	pop 3,45 -> 45+3=48, push	6 48
*	pop 48,6 -> 6*48=288, push	288

Final Result: 288

(This corresponds to the infix expression $6 * (5 + (2 + 3) * 8 + 3)$.)

13. Summary Table of Applications

Application	Stack's Role	Real-World Parallel
Recursion	Stores function call frames (locals, return address)	A chain of sticky notes reminding you what to do next as you go deeper into a task, popped off as you finish each
Balanced Parentheses	Tracks unmatched opening brackets	Matching every "open the door" with a corresponding "close the door", in the correct nested order
Infix -> Postfix	Temporarily holds operators until precedence resolves	A waiter holding orders until it's the right moment to serve them
Infix -> Prefix	Same as above, applied to a reversed & swapped string	Reading the "waiter" trick backwards
Postfix Evaluation	Holds operands until an operator combines them	A calculator's internal working memory
Undo/Redo, Browser History	Stores previous states for LIFO retrieval	Time-travel one step backward at a time
DFS (Depth-First Search)	Explicit or implicit (via recursion) stack tracks the path being explored	Navigating a maze and backtracking when you hit a dead end

14. Practice Problems

Try these to test your understanding:

- Convert to Postfix: $A + B * C - D / E$
- Convert to Prefix: $(A + B) * (C - D) / E$
- Evaluate Postfix: $2\ 3\ 1\ * + 9 -$
- Balanced Check: Is $[(\{ \})]$ balanced? Is $[()]$ balanced?

- Design Challenge: Implement a function `isPalindrome(string)` using two stacks (or one stack and direct comparison).
- Conceptual: Explain why a queue would be a poor substitute for a stack in the "undo" feature of a text editor.
- Recursion -> Iteration: Rewrite a recursive Fibonacci function to use an explicit stack instead of relying on the call stack.
- Design Challenge: Design a stack that supports push, pop, and `getMinimum()` — all in $O(1)$ time. (Hint: use an auxiliary stack.)

Final Takeaway

The stack is deceptively simple — just two operations, push and pop — yet it underlies some of the most important mechanisms in computer science: how your programs execute recursive calls, how compilers parse and evaluate expressions, how editors implement undo, and how algorithms like depth-first search explore graphs. Mastering the stack is a foundational step toward understanding how computation itself is structured.